UserVerificationDocument

Reauthentication Reuse Framework — Application Integration Guide

Field	Details
Audience	Application Development Teams integrating with the Reauthentication Reuse Framework
Version	1.0
Status	Draft for review –> **Team is still updating this document**

Table of Contents

1. Overview

The **Reauthentication Framework** provides a central, standardised capability to reauthenticate a business user. Application teams do **not** implement signing logic themselves — they integrate with this framework by:

1. **Implementing** two interfaces (`IF_ESRA_REAUTH_AUTH_CHECK_PROV` , `IF_ESRA_REAUTH_REL_DATA_PROV`) so the framework can call application specific implementation at the right time.
2. **Calling** one reuse interface (`IF_ESIG_REUSE_API`) at the right points in the appliction business process.

All signature validations (GXP, relevancy, timeout, etag, session conflict checks) are handled centrally inside the framework — the application does not re-implement these.

2. Architecture at a Glance

```
Business Application
├── Implements  →  IF_ESRA_REAUTH_AUTH_CHECK_PROV
                   IF_ESRA_REAUTH_REL_DATA_PROV
                              │          (framework calls back into application)
└── Calls       →  IF_ESIG_REUSE_API           (Application calls into framework)
                        ↓
                   CL_ESIG_REUSE_FACADE        (framework internal — do not reference dire
                   ├── IF_ESIG_RELEVANCY_EVALN  (relevancy checks)
                   ├── IF_ESIG_MANAGE_TIMEOUT   (session timeout handling)
                   ├── IF_ESIG_RTINFO_DAO       (runtime info persistence)
                   ├── IF_ESIG_SERVICE_CLIENT   (backend signing service)
                   └── IF_ESIG_READ_ONLY_DAO    (SOT / field config reads)

Fiori Extension (CDS / RAP)
└── Calls  →  IF_ESIG_FIORI_ADAPTOR           (UI-specific wrapper, NOT IF_ESIG_REUSE_API
                  ↓
              CL_ESIG_REUSE_FACADE             (same facade, shared logic)

WebDynPro Application
└── Calls  →  IF_ESIG_WD_ADAPTORS             (WD wrapper — NOT IF_ESIG_REUSE_API)
              CL_ESIG_WD_ADAPTORS implements:
              ├── REQUEST_REAUTH
              ├── GET_STATUS
              ├── GET_SIGN_HISTORY
              └── UPDATE_BO_OBJECTKEY
                  ↓
              CL_ESIG_REUSE_FACADE             (same facade, shared logic)

SAP GUI Application
└── Calls  →  IF_ESIG_GUI_ADAPTOR             (GUI wrapper — NOT IF_ESIG_REUSE_API)
              CL_ESIG_GUI_ADAPTOR implements:
              ├── REQUEST_REAUTH
              ├── GET_STATUS
              ├── GET_SIGN_HISTORY
              └── UPDATE_BO_OBJECTKEY
                  ↓
              CL_ESIG_REUSE_FACADE             (same facade, shared logic)
```

> **Rule:** Application backend code uses `IF_ESIG_REUSE_API`. Fiori/RAP action handlers use `IF_ESIG_FIORI_ADAPTOR`. WebDynPro applications use `IF_ESIG_WD_ADAPTORS` via `CL_ESIG_WD_ADAPTORS`. SAP GUI applications use `IF_ESIG_GUI_ADAPTOR` via `CL_ESIG_GUI_ADAPTOR`. Never cross these paths.

3. Prerequisites & Configuration

3.1 Reauthentication Object Setup

Before any code is written, the Application team must ensure the following are configured in the Reauthentication tables.

3.1.1 Reauthentication Object

Application team must define a Reauthentication Object based on their business object. This object will glue these 2 applications together. One business object should result in one Reauthentication Object.

- Application must ensure to provide a valid name to a reauthentication object (suggestion equal to your business object). Example PURCHASEORDER, SALESORDER etc.
- Application must also provide a reasonable text so that business users at customer can understand the use of the same.
- Application needs to define 2 classes in the configuration which will be used by the framework.

Note:

- You also have to maintain the same Name with BTP application along with Reason Codes.
- Procedure of BTP Service - Configuration In case of any question on this, please contact Naren.

> [!IMPORTANT] Please remember to provide the details before Sep 16, 2026 for 2702. BTP applicatons can be maintained with later timelines and still to avoid any confusion with dates - we intend to keep the same dates. Although, on case-to-case basis - framework will support the possible changes.

Classes

- <<REAUTH_CLSNAME>>: This class will be used during reauthentication process to check whether the process / change in the document is relevant for reauthentication or not. Application team will ensure , user have authorization for the document to proceed creating signature. If there is an authorization issue , excepton should be sent back to resuable framework to end the process.

- <<AUTHZ_CLSNAME>>: This class will be used during the display of reauthentication logs to check whether user is authorized for this business object - business document or not.

See Section > What Application Must Implement

3.1.2 Reauthentication Object - Node

Reauthentication Node represents an underlying data of a business object for which reauthentication object is defined. Application must define the data hierarchy of their business

object. Using this information, business user should be able to configure the reauthentication relevant field for that business object.

- Application must provide an entity/CDS against each node which should provide an information of fields (also contains customer extensions). (Suggestion: field names shown should be GTNC approved - and have text what customer sees on UI)
- This is a very important configuration as using the given information, fields configuration done by customer - will be validated.

> [!IMPORTANT]
>
> - Only CDS views (STOB) and table names (TABL) are permitted as values for ESRA_SONT-TYPENAME. SQL views should not be used.
> - Be careful on the TYPENAME assignment to the Node. SAP Referene content of Field will freeze in Septemeber. If the assignment of ESRA_SONT-TYPENAME is changed, it must be ensured that the new type contains fields with the same data types as those in the previously used CDS view. The value of TYPENAME is stored only in table ESRA_SONT. Same rule stands true post 2702 after things are delivered to customer.
> - On the F4 of TYPENAME, various options like class names etc comes up. View VC_ESRA_SOT contains design-time system information (system tables), the F4 help for type name has not been restricted exclusively to CDS views and tables. However, users must ensure the correct selection.

3.1.3 Where to maintain?

Transaction SM34 and use view cluster: VC_ESRA_SOT to maintain the information.

3.1.4 SAP Reference Content

Each application have to provide a reference content which will be shipped to customer with scope item 84P.

- Cluster View: VC_ESRA_REL_FIELD
- Table: ESRA_REL_FIELD

As we are making use of the scope item so don't transport entries from this view in the landscape. Content team have prepared template, which must be filled by each business object, once ready send to Xiaorong.

Note:

- Please always remember to provide the template before Scope freeze for features with impact on Best Practice Content Development. For example, Sep 16, 2026 for 2702
- Please always make sure the Reauthentication Object and Reauthentication Object Node Type are released in the landscape.

Maintain Reauthentication Object and Node Type

See: maintain-reauthentication-object-and-node-type

What	Where	How
Reauthentication Object	Configuration	Use SM34 and view cluster: VC_ESRA_SOT
Node Linked to Reauthentication	ESIG customising	Use SM34 and view cluster: VC_ESRA_SOT
Relevancy class registered	`ESRA_SONT` table (`REAUTH_CLSNAME`)	Class must implement `IF_ESIG_RELEVANCY_EVALN`
Authorization class registered	`ESRA_SONT` table (`AUTHZ_CLSNAME`)	Class must implement required auth interface `IF_ESRA_REAUTH_AUTH_CHECK_PROV`

3.2 Runtime Info Table

The framework uses `IF_ESIG_RTINFO_DAO` to persist session state. Application does **not** call this DAO directly — it is managed by the facade. However, Application must ensure the object key structure matches what the framework expects (see `IF_ESIG_API_CALLBACK->UPDATE_BO_OBJECTKEY()`).

3.3 Package Interface

Our development package is ESIG_REAUTH, please make use of package interface assigned to it.

3.4 Factory Access

All framework object instances are obtained via the factory — **never instantiate framework classes directly**:

```
" Correct
DATA(lo_reuse_api) = cl_esig_reuse_factory=>get_esig_facade( ).

" Wrong — do not do this
DATA(lo_facade) = NEW cl_esig_reuse_facade( ).
```

3.5 Request User for IAS Tenant

Currently, the reauthentication framework team is working on the integration aspects with the BTP service. Meanwhile, the framework has already transported a program that can connect to a dedicated BTP tenant, independent of the backend system. This enables application teams to

perform testing and proof-of-concept (POC) activities from any system that can connect to the configured BTP tenant.

To test the end-to-end reauthentication flow, each application team member must request a user account on the GxP IAS (Identity Authentication Service) tenant.

IAS Tenant URL

https://gxp-s4-dev-external.gxp-int-eu12.cfapps.eu12.hana.ondemand.com/

Please use the interactive request form provided below to register users. All entries are stored locally, and Naren (`naren.solanki@sap.com`) is automatically notified whenever a new submission is created.

Notes

- `SOBJType` is required only for the first entry from a team. After the initial submission, the field will automatically be locked for subsequent team members from the same team.
- Since the IAS setup is currently maintained manually, each application team must ensure that their users are added to the IAS tenant so that end users can participate in reauthentication testing.
- Once the communication scenario setup becomes automated, this manual user onboarding activity will no longer be required.

🔒 **Open IAS User Request Form**

4. Developer Onboarding

This section is the fast-path guide for a developer joining an application team that needs to integrate with the Reauthentication Framework.

4.1 What You Need to Know First

Concept	One-line Summary
You do **not** own the sign logic	The framework owns hecks, session locking, BTP calls — you just call the API
Two interfaces to implement	`IF_ESRA_REAUTH_AUTH_CHECK_PROV` (authorization) and `IF_ESRA_REAUTH_REL_DATA_PROV` (relevancy)
One interface to call	`IF_ESIG_REUSE_API` for all backend API calls

Concept	One-line Summary
One factory, one rule	Always use `CL_ESIG_REUSE_FACTORY` — never `NEW cl_esig_...`
Fiori path is different	Fiori/RAP uses the reusable UI library (`IF_ESIG_FIORI_ADAPTOR`), not `IF_ESIG_REUSE_API` directly
Clean up always	Call `DELETE_TEMP_REC()` after every save — success or failure

Note: When the integration with "User Verification" service is active in the system, integrated business applicaitons will call the required framework APIs. These APIs will ensure to put an entry in one Proxy Table in S/4. The entry in this table will act as a lock so that multiple processes can not edit the same document at the same time. **To ensure that transactions keep on working without locks – it is important to delete this entry during the process.**

4.2 Onboarding Steps

```
Step 1 — Understand your BO
  ├── What is the SOT/SONT (SAP Object Name ) for your BO?
  ├── What fields trigger a signature requirement when changed?
  └── Does your BO use Fiori Draft (stateless) or SAPGUI/WD (stateful)?

Step 2 — Verify configuration (do NOT write code until this is confirmed)
  └── Complete your configuration

Step 3 — Implement IF_ESRA_REAUTH_AUTH_CHECK_PROV
  ├── Create requied class (Suggestion: CL_<YOUR_APP>_REAUTH_CHECK)
  └── Implement GET_AUTHORITY_CHECK()  → return true/false based on user authorization for

Step 4 — Implement IF_ESRA_REAUTH_REL_DATA_PROV
  ├── Create required class (Suggestion: CL_<YOUR_APP>_REAUTH_REL_CHK)
  └── Implement GET_RELEVANCY_CHECK()  → return true/false (+ timestamp) if any configured

Step 5 — Add API calls to your application
  Backend path (SAPGUI / WD / headless):
  ├── Sign trigger            → call START_REAUTH()
  ├── After sign dialog       → call VALIDATE_RESULT()
  ├── On late renumbering     → call UPDATE_BO_OBJECT_KEY()
  ├── After COMMIT WORK       → call DELETE_TEMP_REC()


  Fiori / RAP path:
  └── RAP bound action        → use IF_ESIG_FIORI_ADAPTOR via reusable UI library

Step 6 — Go-live checklist (Section 9)
```

4.3 Authorizations

Framework doesn't offer any authorization objects. Framework calls are protected with the application authorizations. **UIs** that declare dependency to the **reuse library** with their manifest, will resolve the required reuse services in their transaction/IAM app.

During the signing process, framwrok will call the application to confirm the relevance for reauthentication. Application must ensure to do the authorization check to ensure that the caller is authorized for the process / object instance.

During the Display Reauthentication Logs. framework have provided an interface wich must be implmeneted by the applicaiton. When such button is clicked in the applicatio UI - applicaiton should call the released framework API. In this API, a callback to the application will be performed. This is to ensure that whether user have required authorizations for the document or not.

4.4 Key Classes at a Glance

What you use	What it does	Remark
`IF_ESIG_REUSE_API`	All backend API calls	`CL_ESIG_REUSE_FACTORY=>get_esig_f` except START_REAUTH()
`IF_ESIG_WD_ADAPTORS`	WebDynpro adaptor	WebDynPro app to integrate and ca REQUEST_REAUTH
`CL_ESIG_GUI_ADAPTOR`	SapGui adaptor integrate and call REQUEST_REAUTH	
`IF_ESRA_REAUTH_AUTH_CHECK_PROV`	Application callback — authorization check	Your own class instance
`IF_ESRA_REAUTH_REL_DATA_PROV`	Application callback — relevancy check	Your own class instance
`IF_ESIG_LOGGER`	Log framework errors	`CL_ESIG_REUSE_FACTORY=>get_esig_l`
`CX_ESIG_ERROR`	Business rule exception — catch it	Raised by framework
`CX_ESIG_EXCEP`	Technical exception —	Raised by framework

What you use	What it does	Remark

*For Fiori integration, please use fiori library *

4.5 Interface IF_ESIG_REUSE_API

Description: Facade Interface for Reauthentication Flow. The primary interface application backend code calls into the framework.

Method	Description
`VALIDATE_RESULT`	Validate signing result and retrieve status
`UPDATE_BO_OBJECT_KEY`	Update object key after late renumbering
`DELETE_TEMP_REC`	Delete temporary records after save
`CREATE_SIGN_MULT_REC`	Create eSignature for multiple records
`UPDATE_SIGN_MULT_REC`	Update eSignature for multiple records

4.5.1 Fiori Integration

See Section 5.2 for full usage details.

4.5.2 `VALIDATE_RESULT` Parameters

Parameter	Direction	Type / Description
`IV_OBJKEY`	Import	Object key of the business document
`IV_SOBJTYPE`	Import	SAP Object Node Type
`IV_RELEVANT`	Import	Relevant status
`IV_MOD_STAMP`	Import	Etag/Timestamp for user verification
`RV_STATUS`	Return	Validation status result

4.5.3 `UPDATE_BO_OBJECT_KEY` Parameters

Parameter	Direction	Type / Description
`IV_SOBJTYPE`	Import	SAP Object Node Type — Camel Case Node Name
`IV_OLD_OBJKEY`	Import	Old Object Key of a Business Document

Parameter	Direction	Type / Description
IV_NEW_OBJKEY	Import	New Object Key of a Business Document
IV_OBJKEY_FIELD1	Import	Object key 1
IV_OBJKEY_FIELD2	Import	Object key 2
IV_OBJKEY_FIELD3	Import	Object key 3
IV_OBJKEY_FIELD4	Import	Object key 4
IV_OBJKEY_FIELD5	Import	Object key 5
EV_PROCESS_STATUS	Export	Status

4.5.4 `DELETE_TEMP_REC` Parameters

Parameter	Direction	Type / Description
IV_SOBJTYPE	Import	SAP Object Node Type
IV_OBJKEY	Import	Object key
RV_SUCCESS	Return	Deletion status

4.5.5 `CREATE_SIGN_MULT_REC` Parameters

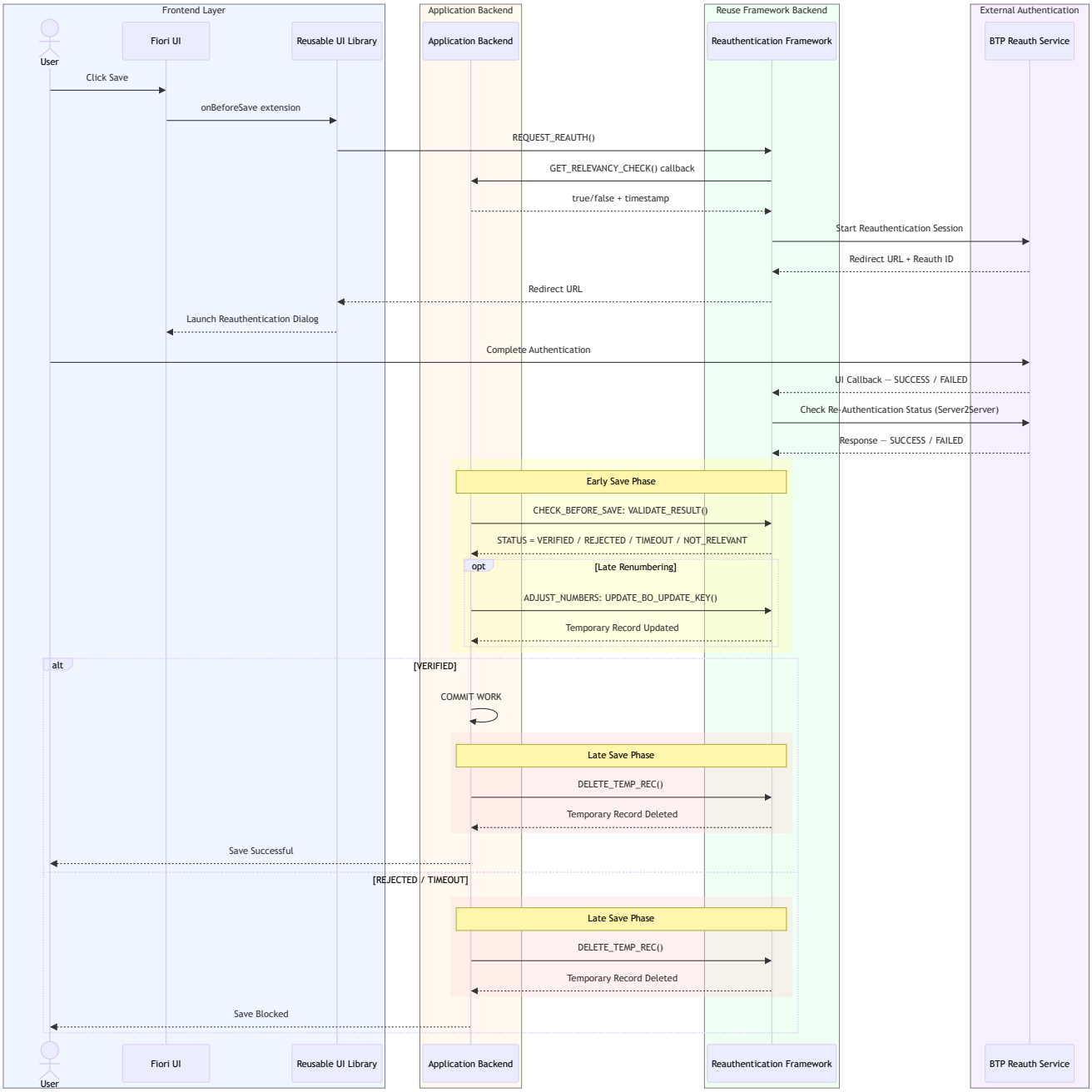
Parameter	Direction	Type / Description
IV_REQUEST	Import	TY_REQ_MULTIPLE — request structure for multiple records
RV_CREATE_RESPONSE	Return	TY_ESIG_CREATE_RESPONSE — creation response

> **Note: `CREATE_SIGN_MULT_REC` is used for bulk/batch signing flows. See Section 5.7 for usage context.**

5. Integration Architecture Flows

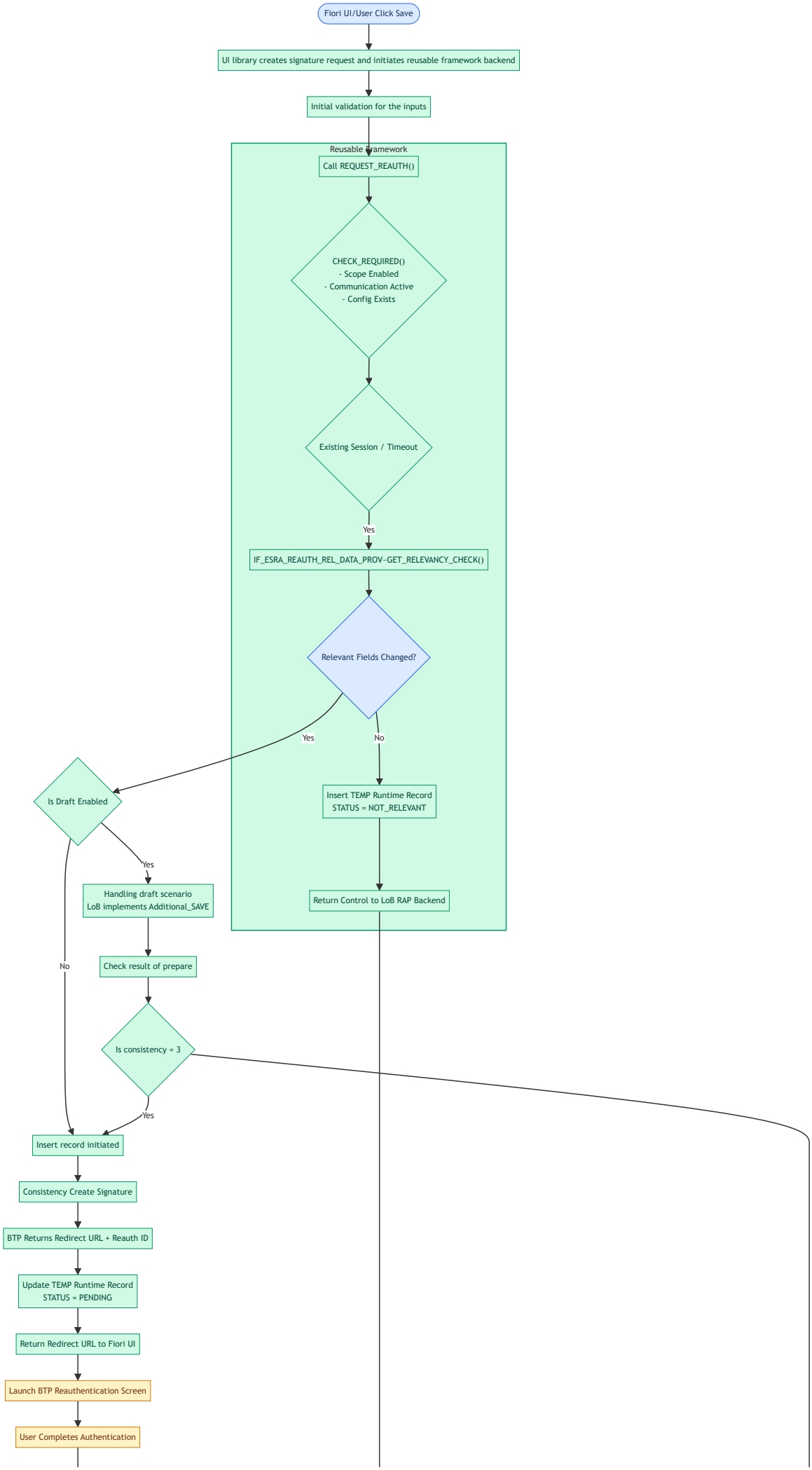
This section explains the end-to-end integration flow between the application, reusable ESIG framework, RAP backend, and BTP reauthentication service.

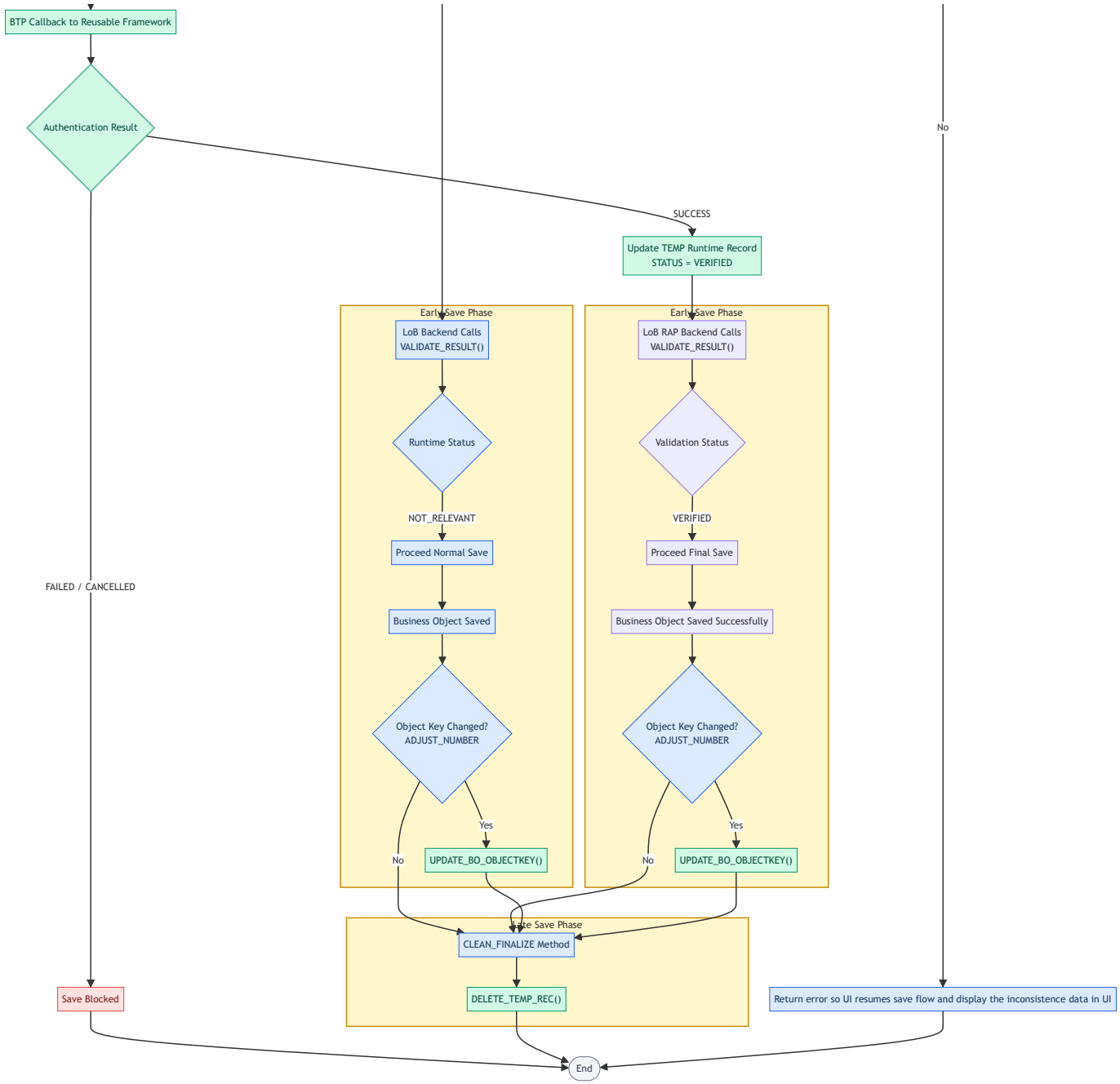
Sequence Diagram



5.1 Create Scenario (Fiori Draft — Stateless)

Colour coding: 🔵 **Blue** — Application	🟢 **Green** — Reusable Framework	🟡 **Amber** — BTP Service	🔴 **Red** — Blocked / Error states

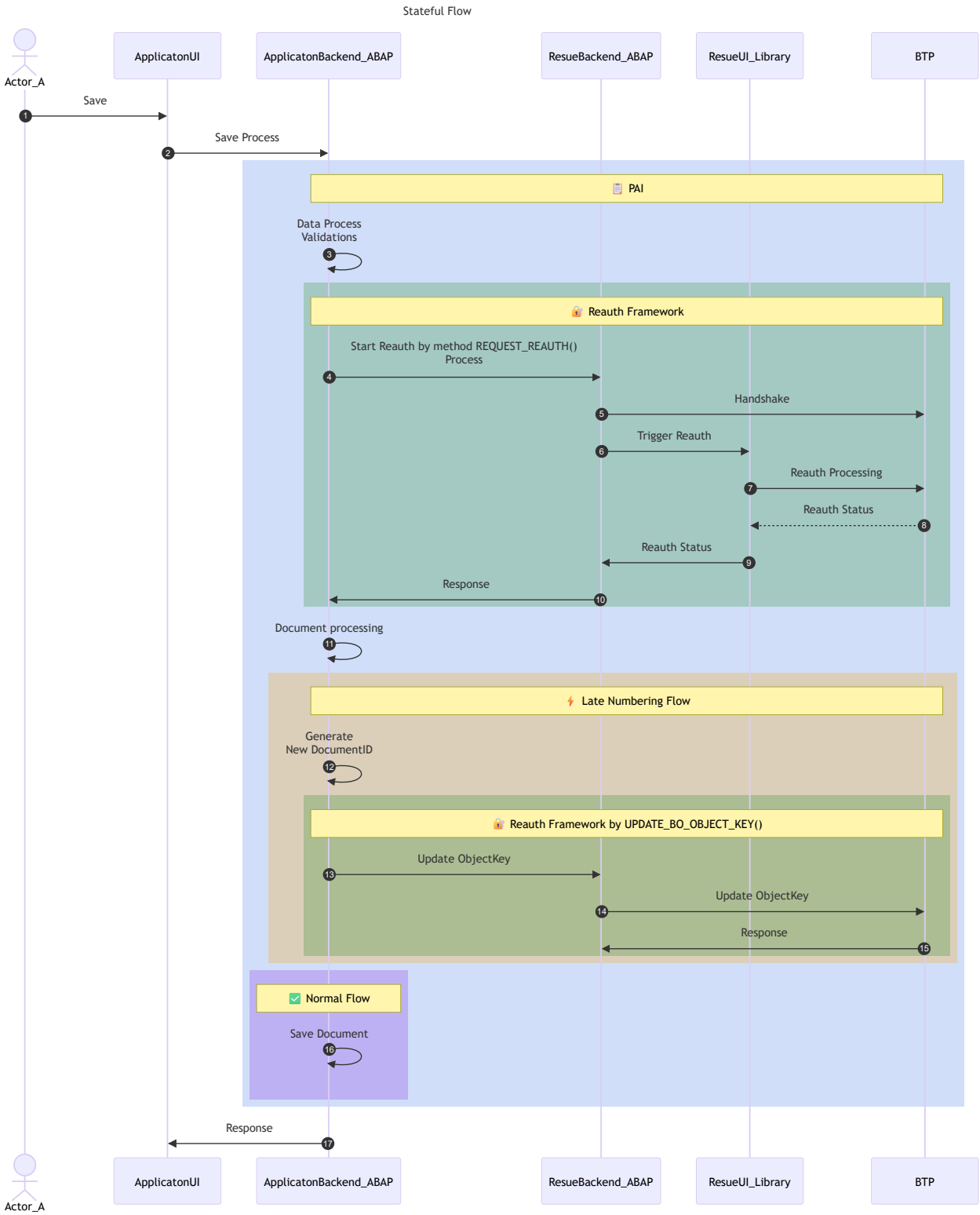

Fiori UI/User Click Save
UI library creates signature request and initiates reusable framework backend
Initial validation for the inputs
Reusable Framework
Call REQUEST_REAUTH()
CHECK_REQUIRED()
- Scope Enabled
- Communication Active
- Config Exists
Existing Session / Timeout
Yes
IF_ESRA_REAUTH_REL_DATA_PROV~GET_RELEVANCY_CHECK()
Relevant Fields Changed?
Yes
No
Is Draft Enabled
Yes
Handling draft scenario
LoB implements Additional_SAVE
Check result of prepare
Is consistency = 3
Yes
No
Insert TEMP Runtime Record
STATUS = NOT_RELEVANT
Return Control to LoB RAP Backend
Insert record initiated
Consistency Create Signature
BTP Returns Redirect URL + Reauth ID
Update TEMP Runtime Record
STATUS = PENDING
Return Redirect URL to Fiori UI
Launch BTP Reauthentication Screen
User Completes Authentication



5.2 Save / Change Scenario (SAPGUI / WebDynpro — Stateful)

This is the stateful path used by SAPGUI and WebDynpro applications. The application calls the framework API directly from the ABAP backend — there is no Fiori UI library involved. Reauthentication is triggered explicitly via `REQUEST_REAUTH()` using the adaptor class, and the save can only proceed once reauthentcation is confirmed. This API should only be integrated if reauthentication is active in the system.

Sequence Diagram



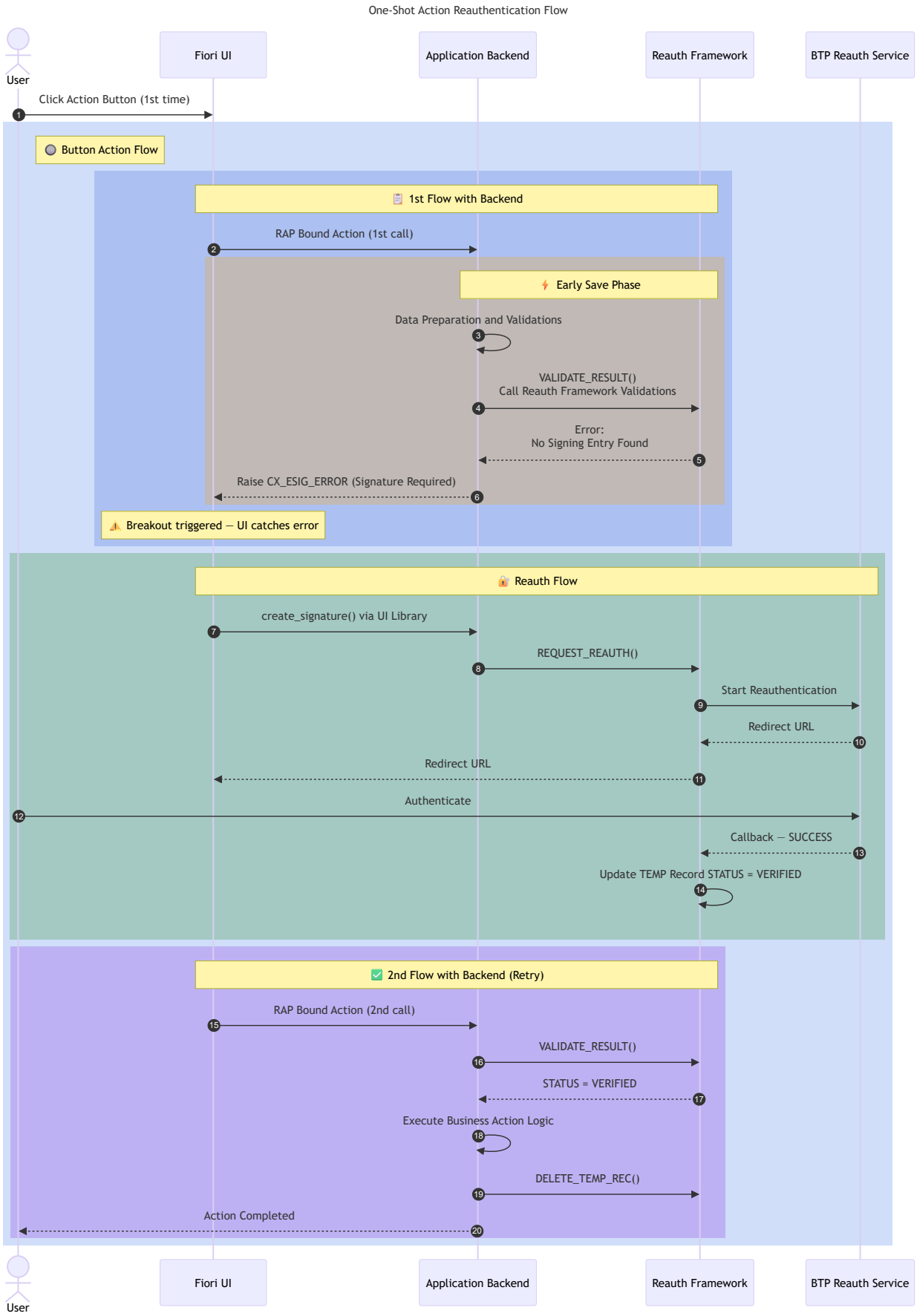
5.3 Fiori 1-Shot Action — Breakout Pattern

Static Action / One-Shot Actions

A **static action** (also called a one-shot action) is a bound RAP action the user deliberately triggers — e.g. “Approve”, “Release”, “Override” — that requires reauthentication before executing. Unlike the save flow, there are no changed fields to evaluate; the action itself is the trigger.

The key pattern is the **breakout**: the first call deliberately throws an error (no proxy exists yet), the UI catches it, drives reauthentication, then retries.

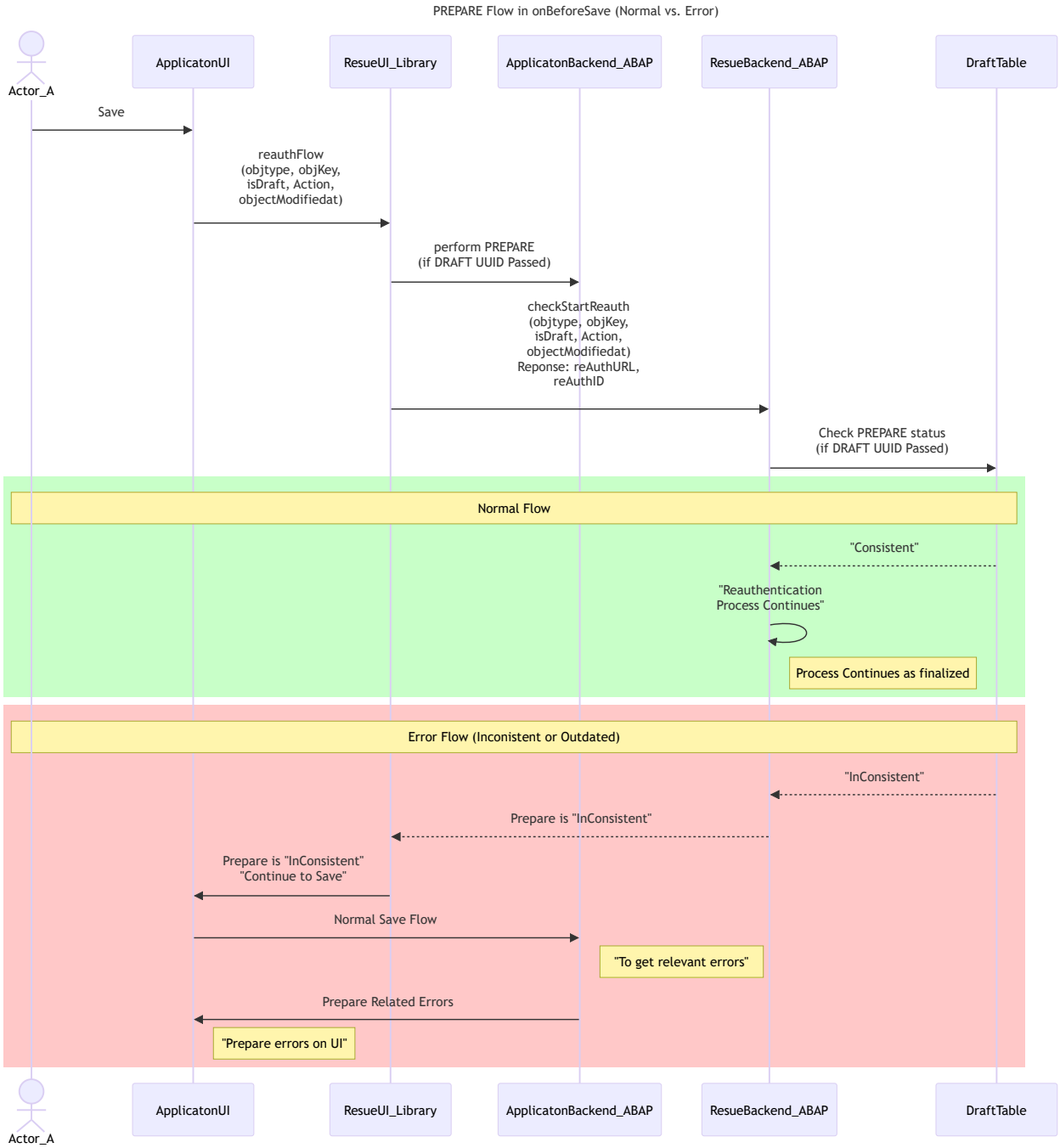
Sequence Diagram



5.4 Prepare Flow

Applications working with Draft setup, should pass the administrative draft UUID to the Reuse framework. This is important as framework will call the PREPARE ACTION. Prepare action is called to ensure that user doesn't reauthenticate to the scenarios where draft is not yet consistent.

Flow for the prepare scenario will look like this.



5.5 Relevance and LastModifiedat during CheckBeforeSave

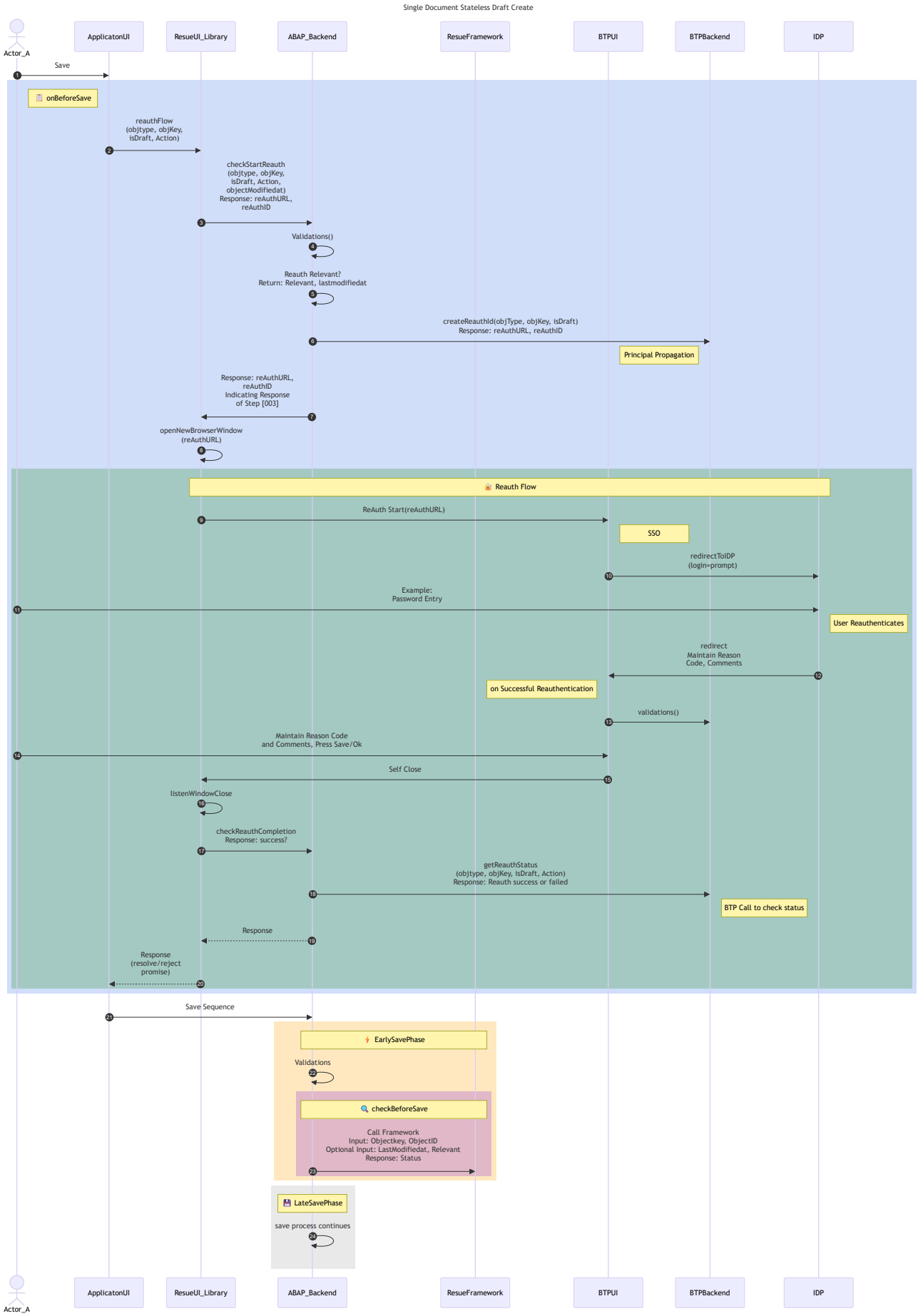
It is important to understand from the framework point of view that on what state of the document, reauthenticaiton is perfomed. This parameter is to indicate when the document was last changed (like last eTAG). We are supporting currently a last timestamp.

For this a parameter is provided in the callback method of class - «REAUTH_CLSNAME» along with instance relevance indicator. This value will be stored in the ProxyTable of the framework. These values are provided to the framework during “onBeforeSave”.

Applicaiton can also decide to re-check whether the document is reauthentication relevant or not.

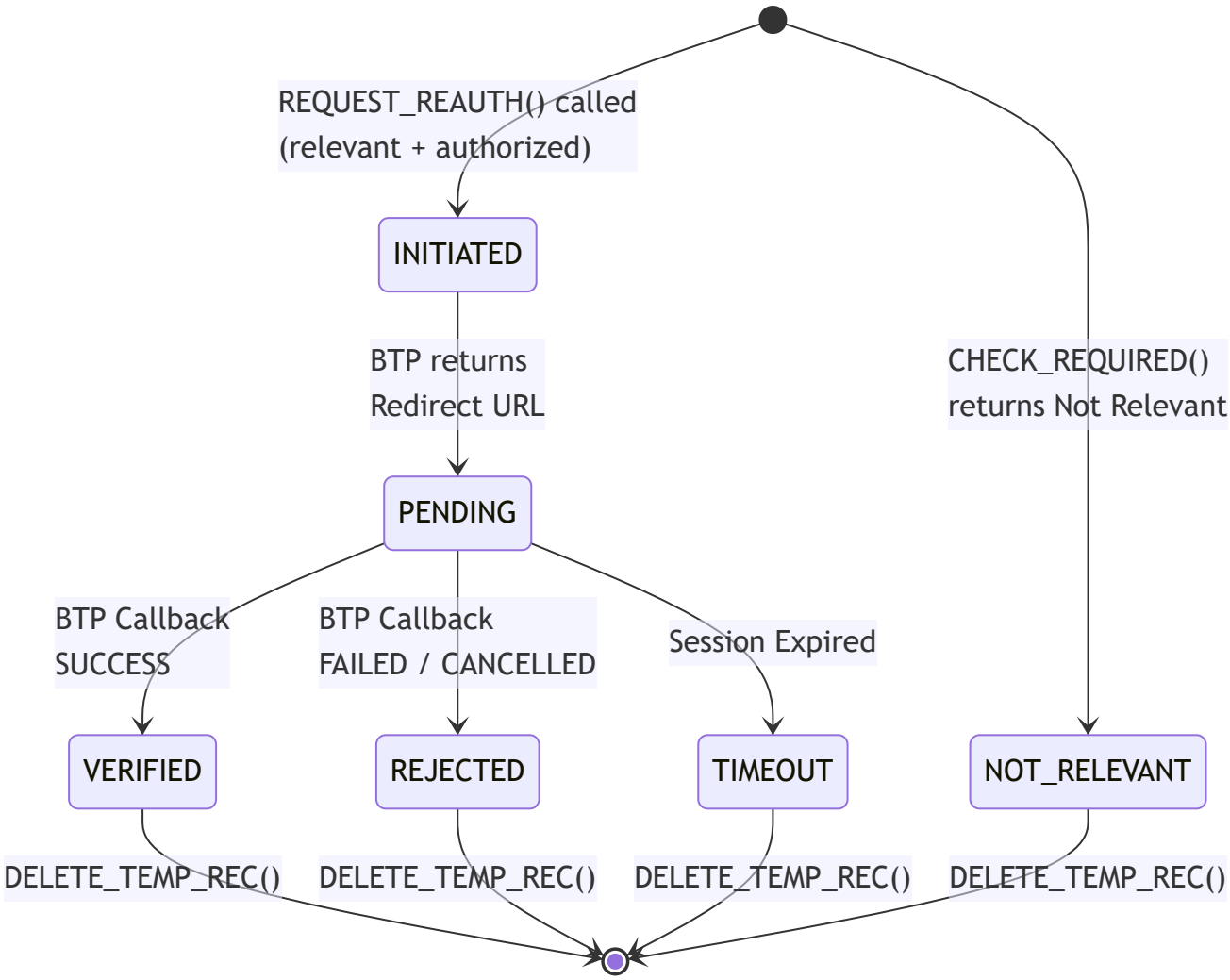
As mentioned above also, during “checkBeforeSave” - applicaiton must call “validate_result”. In this method, 2 values can be sent optionally (lastmodifiedtime, relevance). Using these 2 values, framework will check the state of the document (stored in the DB) when the reauthenticatoon were performed during “onBeforeSave”.

Below is a sequence diagram to highlight this part.



5.6 State Diagram of Local Reauth Proxy Table

The framework tracks each signing session via a temporary runtime record. Below are all possible state transitions:



6. What Application Must Implement — `IF_ESRA_REAUTH_AUTH_CHECK_PROV` & `IF_ESRA_REAUTH_REL_DATA_PROV`

The LoB team must create classes implementing both callback interfaces. The framework calls these classes to retrieve LoB-specific context during the signing flow.

6.1 `IF_ESRA_REAUTH_AUTH_CHECK_PROV` — Authorization Check

Implement `GET_AUTHORITY_CHECK()` to return whether the current user is authorized to sign the given business object. Perform your LoB-specific authority check here.

6.2 `IF_ESRA_REAUTH_REL_DATA_PROV` — Relevancy Check

Implement `GET_RELEVANCY_CHECK()` to compare the changed fields on the BO against the fields configured as signature-relevant in the ESIG customising. If **any one** configured field has changed, return `abap_true` along with the timestamp of the change.

> **Important:** Register your callback class instances with the facade before calling any `IF_ESIG_REUSE_API` method.

7. What Application Calls — `IF_ESIG_REUSE_API`

7.1 Start Authentication

When to call:

- **Fiori / RAP path:** The user clicks Save → the reusable UI library handles `CHECK_REQUIRED()` ` automatically. LoB does not call this directly; instead, invoke the library as described in the integration guide.
- **SAPGUI / WD / headless path:** Call `REQUEST_REAUTH()` to launch the signing session.

Integrate for Start Reauthentication

See: Integrate_CreateSignature_Reuse_Library.md

What it checks internally:

#	Check	Detail
1	Scope active?	Is the ESIG scope enabled for this SONT?
2	Integration scenario active?	Is the specific signing scenario configured?
3	Relevancy / Auth-Class present?	`ESRA_SONT` must have both class entries and LoB logic must be provided
4	Callback: user authorized?	Framework calls `IF_ESRA_REAUTH_AUTH_CHECK_PROV~GET_AUTHORITY_CHECK()`
5	Collaborative draft etag match?	For draft objects: current etag must match session etag to prevent simultaneous editing conflicts

> **Etag rule:** For collaborative draft scenarios, always pass the current etag of the draft object. A mismatch means another user has modified the object — the framework will reject the authentication start to prevent conflicting signatures.

7.2 ValidateResult — `VALIDATE_RESULT()`

When to call: ` — to retrieve the final outcome before allowing the save to proceed. This is valid for the statesless calls during CheckBeforeSave.

Statuses returned:

Status	Meaning	Applicaiton Action
INIT	Initiated	Do not Proceed with save
NOREL	Not Relevant for Reauthentication	Proceed with Save
NOMHR	Not Matching Reauthentication Status	Do not Proceed with Save
NOMHT	Not Matching Reauthentication LastModTime	Do not Proceed with Save
SUCCS	Reauthentication Successful	Proceed with Save

> **Note:** More content to be added.

Always call `VALIDATE_RESULT()` . The framework requires this call to confirm the runtime record state before the save is permitted.

7.3 GetHistory — `GET_SIGN_HISTORY()`

When to call: Whenever the LoB UI or backend needs to display signature history for a BO (e.g. history tab, audit report).

Integrate for User Verification Logs

See: Integrate_GetHistory_Reuse_Library.md

Fields available in each history record:

Field	Description
`SIGNEDON`	Timestamp of signing
`SIGNEDBY`	User who signed
`STATUS`	Result (`SIGNED` / `REJECTED`)
`REASON`	Reason code
`COMMENTS`	Free-text signer comment

7.4 UpdateObjectID — Late Renumbering — `UPDATE_BO_OBJECT_KEY()`

When to call: After the new permanent BO key is assigned during the final save — once the key is known and before any subsequent signing flows reference the same object. This is a backend-to-backend call — no user interaction involved.

The framework will:

1. Update its internal runtime info table with the new key.
2. Call back `IF_ESIG_API_CALLBACK->UPDATE_BO_OBJECTKEY()` on the LoB callback so the LoB can mirror the key update in its own tables.

7.5 DeleteTempRec — Post-Save Cleanup — `DELETE_TEMP_REC()`

When to call: After a **final save completes** — whether successful or failed — to clean up any temporary runtime records created during the signing session.

```
DATA(lo_api) = cl_esig_reuse_factory=>get_esig_facade( ).

TRY.
    lo_api->delete_temp_rec(
      iv_object_id   = lv_bo_object_id
      iv_object_type = lv_sont
    ).

  CATCH cx_esig_excep INTO DATA(lx_technical).
    " Log but do not block — cleanup failure is non-critical at this stage
    cl_esig_reuse_factory=>get_esig_logger( )->log_exception( lx_technical ).
ENDTRY.
```

> **Always call this**, even on save failure. Orphaned temporary records cause false session-conflict errors on the next save attempt.

7.6 Bulk Processing — `CREATE_SIGN_MULT_REC()` & `UPDATE_RESULT_MULT_REC`

> **Note:** to be added.

8. Fiori Extension via CDS / RAP

If the LoB is building a Fiori UI using CDS consumption views and RAP, the signing trigger comes from a **RAP action** — not directly from backend ABAP. Use the reusable UI library, which internally uses `IF_ESIG_FIORI_ADAPTOR`. See the library integration guide for setup instructions.

Integrate for Start Reauthentication

See: Integrate_CreateSignature_Reuse_Library.md

Integrate for User Verification Logs

See: Integrate_GetHistory_Reuse_Library.md

Rule: Use the standard framework reusable library, not `IF_ESIG_REUSE_API`

9. Error Handling

The framework raises two exception classes. LoB code must handle both:

Exception Class	Type	When Raised
`CX_ESIG_EXCEP`	Final (non-resumable)	Unexpected / technical errors (HTTP failure, service unavailable, data inconsistency)
`CX_ESIG_ERROR`	Final (non-resumable)	Business rule violations (no auth class, relevancy mismatch, invalid status)

Both inherit from `CX_ROOT` via `CX_STATIC_CHECK`.

Handling Pattern

```
DATA(lo_api)    = cl_esig_reuse_factory=>get_esig_facade( ).
DATA(lo_logger) = cl_esig_reuse_factory=>get_esig_logger( ).

TRY.
    lo_api->start_reauth(
      io_auth_check_prov = mo_my_auth_check_impl
      iv_object_id       = lv_bo_object_id
      iv_object_type     = lv_sont
      iv_etag            = lv_current_etag
    ).

  CATCH cx_esig_error INTO DATA(lx_business).
    " Business rule violation — surface the framework message directly to the user
    MESSAGE lx_business->get_text( ) TYPE 'E'.

  CATCH cx_esig_excep INTO DATA(lx_technical).
    " Technical / infrastructure error — log full details, show generic message
    lo_logger->log_exception( lx_technical ).
    MESSAGE 'Electronic signature service is temporarily unavailable. Please try again or
ENDTRY.
```

Key Exception Constants to Expect

Constant	Meaning	LoB Action
`CX_REAUTH_NOT_ALLOWED`	Signing not allowed for this object/user	Surface to user, do not retry automatically
`CX_RETRIEVE_STATUS_FAILED`	Cannot read current session status	Retry once, then escalate
`CX_INVALID_DOC_TYPE`	SONT not configured	Raise with ESIG Basis team — config issue
`CX_NO_AUTHORITY`	User lacks authorization	Surface to user
`CX_REAUTH_STATUS_UNDETERMINED`	Status indeterminate after signing	Do not commit save — prompt user to retry
`HTTP_*` constants	Network / service errors	Log and surface generic message

10. Development Checklist

Use this checklist during development and peer review.

Configuration

- ☐ SOT created and activated for the LoB object type
- ☐ SONT linked to SOT and configured correctly
- ☐ Relevancy class (`REAUTH_CLSNAME`) registered in `ESRA_SONT`
- ☐ Authorization class (`AUTHZ_CLSNAME`) registered in `ESRA_SONT`

Callback Implementation (`IF_ESRA_REAUTH_AUTH_CHECK_PROV` , `IF_ESRA_REAUTH_REL_DATA_PROV`)

- ☐ `GET_AUTHORITY_CHECK()` returns `true` / `false` based on whether the current user is authorized to sign the object
- ☐ `GET_RELEVANCY_CHECK()` returns `true` / `false` with timestamp if any configured field has changed

API Calls (`IF_ESIG_REUSE_API` , `IF_ESIG_WD_ADAPTORS` , `IF_ESIG_GUI_ADAPTOR`)

- ☐ `REQUEST_REAUTH()` starting point for starting reauthentication for WD,SAPGUI`
- ☐ Etag passed to for all collaborative/draft scenarios (refer UI library integrate for Start Reauthentication)
- ☐ `VALIDATE_RESULT()` called after signing dialog returns — all four statuses handled (`COMPLETED`, `REJECTED`, `TIMEOUT`, `NOT_RELEVANT`)
- ☐ `DELETE_TEMP_REC()` called in both save-success and save-failure paths
- ☐ `UPDATE_BO_OBJECT_KEY()` called only after late renumbering, after the new key is assigned
- ☐ Factory used for all framework object access — no direct class instantiation

Error Handling

- ☐ Both `CX_ESIG_EXCEP` and `CX_ESIG_ERROR` caught at all call sites
- ☐ Technical errors logged via `IF_ESIG_LOGGER`
- ☐ Business errors surfaced to user with meaningful messages
- ☐ No signing check logic duplicated in LoB code

Fiori / CDS (if applicable)

- ☐ Follow the reusable library

11. Go-Live Checklist

- ☐ End-to-end signing flow tested in quality system with real user credentials
- ☐ Bulk processing tested with minimum 50 records
- ☐ Timeout scenario tested: session created, left to expire, new signing initiated — no orphan record errors
- ☐ Collaborative draft scenario tested: two users editing same draft simultaneously — second user correctly blocked
- ☐ Late renumbering tested: preliminary key → final key, history retrieval works on final key
- ☐ `DELETE_TEMP_REC()` on save failure tested — no orphan records in RTINFO table after failed save
- ☐ Authorization check tested with user lacking signing authority — correct error surfaced
- ☐ GXP inactive scenario tested (if applicable) — no signature prompted
- ☐ Performance: `CHECK_REQUIRED()` benchmarked in mass-save scenario (background job with many BOs)
- ☐ Transport of SONT / SOT configuration included in release transport
- ☐ Config content review by ESIG framework Team before finalization
- ☐ ESIG framework team sign-off obtained before production deployment

12. FAQ

Q: Can we call `IF_ESIG_SERVICE_CLIENT` directly instead of `IF_ESIG_REUSE_API` ? No. `IF_ESIG_SERVICE_CLIENT` is an internal framework interface connecting to the backend signing service. It does not perform the business rule checks (GXP, relevancy, session conflict) that `IF_ESIG_REUSE_API` provides. Direct use will bypass all safety checks.

Q: What if the etag changes between `CHECK_REQUIRED()` and `REQUEST_REAUTH()` ? This is an expected race condition in collaborative draft scenarios. `REQUEST_REAUTH()` will raise `CX_ESIG_EXCEP` with a log-header-inconsistent-type error. Surface a message asking the user to refresh and retry — do not auto-retry silently.

Q: Can the same callback class instance be shared across multiple BOs in a bulk loop? No. Instantiate a fresh callback per BO in bulk processing, as each instance carries BO-specific context (changed fields, current status). Sharing an instance risks state bleed between BOs.

Q: After `DELETE_TEMP_REC()` , can we still call `GET_HISTORY()` ? Yes. `DELETE_TEMP_REC()` removes the in-progress session runtime record only. The permanent signature history stored via `IF_ESIG_SERVICE_CLIENT` is unaffected and always available via `GET_HISTORY()` .

Q: Who do we contact for ESIG framework issues? Raise an incident to the **CA-DMI-REA** component with: SONT name, object ID (anonymised if needed), timestamp, and the full exception text from `CX_ESIG_EXCEP` / `CX_ESIG_ERROR` .

12.1 Where to Call — Quick Reference

Integration Points and Framework APIs

The following integration points are used by the Reauthentication Framework to support relevancy validation, authorization checks, runtime session handling, audit logging, and cleanup activities.

Integration Point	Interface / Method	Purpose
Relevancy Evaluation	`IF_ESRA_REAUTH_REL_DATA_PROV~GET_RELEVANCY_CHECK()`	Determines whether the modified business fields are configured as reauthenticatic relevant fields.

Integration Point	Interface / Method	Purpose
Authorization Validation	`IF_ESRA_REAUTH_AUTH_CHECK_PROV~GET_AUTHORITY_CHECK()`	Validates whether the current user is authorized to perform the business actio requiring reauthenticati
Start Reauthentication (Fiori)	Refer to Fiori Reauthentication Flow section	Launches the reusable Fiori reauthenticati library and initiates the er user verificatic process.
Validate Result	`IF_ESIG_REUSE_API~VALIDATE_RESULT()`	Validates the final authentication result before t business objec save operatior completed.
Update Business Object Key	`IF_ESIG_REUSE_API~UPDATE_BO_OBJECT_KEY()`	Updates the fi business objec key in runtime records for scenarios involving late numbering or temporary obj IDs.
Cleanup Runtime Record	`IF_ESIG_REUSE_API~DELETE_TEMP_REC()`	Removes temporary runtime/sessic records after successful completion or

Integration Point	Interface / Method	Purpose
		termination of the process.
Retrieve Audit History	`IF_ESIG_REUSE_API~GET_SIGN_HISTORY()`	Retrieves authentication and signing audit history associated wit the business

Additional Notes

- The framework supports both stateless Fiori draft flows and backend-triggered save validations.
- Runtime records are maintained temporarily until the save transaction is completed successfully.
- In late numbering scenarios, the framework updates temporary object references with the final persisted business object key.
- Audit history APIs can be consumed for compliance reporting, monitoring, or traceability requirements.

- **The reusable APIs are designed to support multiple LoB applications with a centralized authentication and audit mechanism.**

12.3 Important Design Rules

1. LoB backend applications must only consume `IF_ESIG_REUSE_API`.
2. Fiori / RAP applications must use the reusable UI library (which wraps `IF_ESIG_FIORI_ADAPTOR`) — do not call `IF_ESIG_REUSE_API` directly from RAP action handlers.
3. Never instantiate framework classes directly using `NEW`.
4. Always use `CL_ESIG_REUSE_FACTORY`.
5. Always call `DELETE_TEMP_REC()` after save completion — success or failure.
6. For draft-enabled applications, always pass the current ETag to `framework`.
7. Runtime status `NOT_RELEVANT` must still go through `VALIDATE_RESULT()` before final save.

13. Testing Landscape

This section explains where and how LoB teams can test their ESIG integration.

13.1 System Landscape

System	Purpose	BTP Connected	Notes
DEV	Unit development and initial integration testing	No	Use mock / stub for BTP signing; `IS_REAUTH_ACTIVE()` may return false
QA / Integration	End-to-end flow testing with real BTP signing	Yes	Full flow including BTP dialog and callback
Pre-Production	Final validation before go-live	Yes	Use anonymised / test data only
Production	Live system	Yes	No testing — incidents only

> **DEV systems:** Since BTP is not connected in DEV, use `IS_REAUTH_ACTIVE()` as a guard to skip the signing flow and test the LoB logic independently.

13.2 What to Test

Functional Scenarios

Scenario	Where	Pass Criteria
Relevant field changed → signing triggered	QA	`CHECK_REQUIRED()` returns `gc_check_required`, dialog appears
No relevant field changed → signing skipped	QA	`CHECK_REQUIRED()` returns `gc_check_not_relevant`, save proceeds normally
User completes signing → save allowed	QA	`VALIDATE_RESULT()` returns `VERIFIED`, object saved successfully
User cancels signing → save blocked	QA	`VALIDATE_RESULT()` returns `REJECTED`, user sees error message
Session timeout → save blocked	QA	`VALIDATE_RESULT()` returns `TIMEOUT`, user prompted to retry

Scenario	Where	Pass Criteria
Authorization failure → signing blocked	QA	`GET_AUTHORITY_CHECK()` returns false, `CX_ESIG_ERROR` raised
Late renumbering → key updated in framework	QA	`GET_HISTORY()` returns history on new permanent key
`DELETE_TEMP_REC()` on save failure → no orphan	QA	No session-conflict error on subsequent save attempt

Boundary Scenarios

Scenario	Where	Pass Criteria
Two users editing same draft simultaneously	QA	Second user correctly blocked with etag conflict error
Signing session left to expire, then new save	QA	Previous timeout record cleaned up, new signing session starts cleanly
Bulk processing (min. 50 records)	QA	All results returned, no partial failures or session leaks
GxP flag inactive (if applicable)	QA	No signing dialog triggered, save proceeds normally
`IS_REAUTH_ACTIVE()` = false	DEV	Entire signing flow skipped gracefully

13.3 Test Checklist

- ☐ End-to-end flow tested in QA with real user credentials (not service user)
- ☐ Signing dialog renders correctly in target browser / Fiori launchpad
- ☐ BTP callback received and TEMP record updated to VERIFIED
- ☐ `DELETE_TEMP_REC()` on save failure — no orphan records in RTINFO table
- ☐ `UPDATE_BO_OBJECT_KEY()` tested for late renumbering scenario
- ☐ Bulk processing tested with ≥ 50 records
- ☐ Timeout and rejection paths tested
- ☐ Concurrent user test (etag conflict) passed
- ☐ Transport of SONT / SOT configuration included in release transport
- ☐ ESIG framework team sign-off obtained before production deployment

14. Delivery Timelines

> Dates are anchored to the **2702 release**. Update the `<<PLACEHOLDER>>` entries once milestones are confirmed.

Milestone	Date	Details
Scope Freeze (2702)	**16 Sep 2026**	No new reauthentication-relevant fields or business objects may be added after this date. Existing integrations must be code-complete.
Content Review Deadline	`<<DATE>>`	SAP Reference Content template (Section 3.1.4) must be submitted to Xiaorong before this date.
Feature Freeze	`<<DATE>>`	All API integration code (`VALIDATE_RESULT` , `DELETE_TEMP_REC` , callback classes) must be merged and tested in the quality system.
ESIG Framework Sign-off	`<<DATE>>`	Go-live checklist (Section 11) must be completed and reviewed by the ESIG framework team before this date.
Production Deployment	`<<DATE>>`	No reauthentication-related transports after this cutoff without an approved emergency change request.

14.1 What Can Change After Scope Freeze

Change Type	Allowed After Freeze?	Process
Bug fix to existing callback class	Yes	Raise a PR, get framework team review, standard transport
New Reauthentication-relevant field	**No**	Must wait for next release cycle
New Reauthentication object	**No**	Must wait for next release cycle
Emergency fix to a production blocker	Yes — restricted	Framework team to evaluate
Transport of Reauth configuration	Yes — until production cutoff	Must be included in release transport before cutoff date

15. Contributing & GitHub Workflow

This document is maintained in GitHub. Use the processes below to ask questions, report issues, or propose changes.

15.1 Opening a Discussion (Questions About Content)

Use GitHub Discussions when you have a question about the integration guide — not a bug or a concrete change request.

1. Go to the Discussions tab of this repository.
2. Click **New Discussion** and select the category **Q&A**.
3. Title format: `[<Your BO / Team>] <Short question summary>` — for example:
   `[PurchaseOrder] When exactly should DELETE_TEMP_REC be called on a failed draft save?`
4. In the body, include:
   - Your business object / SONT name
   - The section of this document the question relates to
   - What you have already tried or checked
5. Apply the label `question` (see Section 15.2).

> Discussions are preferred over email — the answer is then visible to all integration teams.

15.2 Adding Labels to Issues and Pull Requests

To browse existing issues or open a new one, go to the Issues tab.

Apply labels when opening an issue or PR so the framework team can triage quickly.

Label	When to Use
`question`	You need clarification on documented behaviour — no code change implied
`bug`	Documented behaviour does not match actual framework behaviour
`content`	Change to this integration guide (wording, examples, diagrams)
`config`	Change related to SONT / SOT / VC_ESRA_SOT configuration guidance
`emergency`	Production blocker requiring fast-track review — Framework to Evaluate
`enhancement`	Suggested improvement to the guide or framework API
`wontfix`	Acknowledged but out of scope for current release — added by framework team

To add a label: on the issue or PR page, click the **Labels** gear icon on the right-hand panel and select the appropriate label(s). If a label you need does not exist, contact the framework team to have it created — do not improvise label names.

15.4 Subscribe to Release Notifications

When the framework team publishes a new version of the integration guide, GitHub sends a notification to all subscribers. Follow the steps below to ensure you are notified whenever released files are updated.

Step-by-step

1. Open the repository: https://github.tools.sap/I311077/UserVerificationDocument
2. Click the **Watch** button at the top-right of the page (next to Star and Fork).
3. A dropdown appears — select **Custom**.
4. In the custom options, check **Releases** only.
5. Uncheck everything else (Issues, Pull Requests, Discussions, etc.) to avoid unrelated noise.
6. Click **Apply**.

You will now receive a notification (email or GitHub notification, depending on your GitHub profile settings) each time a new release is published.

What a release notification contains

Each release notification includes:

- A version tag (e.g. `v1.1`)
- A summary of which files changed and what was updated
- A direct link to the full release notes
- Links to open a Discussion or Issue if you have questions about the changes

Check your GitHub notification settings

If you are not receiving emails after subscribing:

1. Go to your GitHub profile → **Settings** → **Notifications**
2. Under **Participating and watching**, ensure **Email** is checked
3. Confirm your email address is verified under **Settings** → **Emails**

> **Recommendation for LoB teams:** At minimum, one developer and one tech lead per application team should subscribe to releases. This ensures your team is always working from the latest integration guide.

Direct link to all releases

https://github.tools.sap/I311077/UserVerificationDocument/releases

*Document maintained by the Reauthentication Framework Team. For updates or corrections, raise an internal incident to component **CA-DMI-REA**. & Team DL*

DL_692FCB401D1A776CA2DD975F@global.corp.sap For any urgent issues needs attention, please tag I058982 (Naveen Kumar RC).